BRAINWARE UNIVERSITY
398, Ramkrishnapur Road, Barasat, North 24 Parganas, Kolkata - 700 125

LEARNING RECOMMENDER SYSTEM

A PROJECT REPORT

Submitted by

Roll Number	Registration Number	Student Code	Student Name
24103460061	24103002842	BWU/BCA/24/063	Priya Naskar

in partial fulfilment for the award of the degree

of

BACHELOR OF COMPUTER APPLICATIONS (HONS)

In

DEPARTMENT OF COMPUTATIONAL SCIENCES

BRAINWARE UNIVERSITY
398, Ramkrishnapur Road, Barasat, North 24 Parganas,
Kolkata - 700 125

November (2026-2027)



BRAINWARE UNIVERSITY

398, Ramkrishnapur Road, Barasat, North 24 Parganas, Kolkata - 700 125

DEPARTMENT OF COMPUTATIONAL SCIENCES

BONAFIDE CERTIFICATE

Certified that the project report titled “LEARNING RECOMMENDER SYSTEM” is the Bonafide work of “PRIYA NASKAR” (Roll Number: 24103460061, Student Code: BWU/BCA/24/063) , carried out in partial fulfilment of the requirements for the Bachelor of Computer Applications (Hons) degree, under the supervision of the faculty member assigned by the Department of Computational Sciences.

SIGNATURE	SIGNATURE
DR. JAYANTA AICH RAAHUL KUMAR GHOSH	DR.
HEAD OF THE DEPARTMENT ASSOCIATE PROFESSOR	
COMPUTATIONAL SCIENCES COMPUTATIONAL SCIENCES	
BRAINWARE UNVERSITY BRAINWARE UNVERSITY	

ACKNOWLEDGEMENT

I would like to express my profound gratitude to all those who have been instrumental in the successful completion of this project report. Their continuous support and encouragement have played a vital role in shaping my learning experience.

I extend my sincere thanks to our esteemed project supervisor, **Dr. Raahul Kumar Ghosh**, **Associate Professor**, whose constant guidance, valuable suggestions, and insightful feedback have been instrumental throughout the development of this project. His expertise, patience and motivation helped us overcome challenges and understand the core concepts effectively.

I also deeply thankful to **Dr. Jayanta Aich**, **Head of the Department of Computational Sciences**, for providing a supportive academic environment and the necessary facilities that enabled us to complete this work successfully.

My sincere appreciation goes to all the faculty members of the Department of Computational Sciences for sharing their knowledge, advice, and encouragement during my project. Their teachings have significantly contributed to strengthening my technical understanding and skills.

Finally, we express my profound gratitude to our families and friends, whose constant support, understanding, and motivation inspired us to stay committed and focused during every phase of this project. Without their encouragement, this achievement would not have been possible.

SIGNATURE :-

ABSTRACT

The Learning Recommender System is a machine-learning application that ranks next-course recommendations from learner profile data. The Streamlit dashboard accepts either an existing learner ID or a manually entered learner profile, compares the profile with similar learners, and presents recommended course codes with match scores and supporting explanations.

The project uses data/learning_path_dataset.csv as its source dataset. The data contains learner attributes and recorded next-course labels. It does not contain individual learning-material identifiers, resource descriptions, activity types, or resource URLs; therefore, the system recommends courses rather than videos, quizzes, or other learning materials.

The recommendation engine uses scikit-learn's DictVectorizer to transform categorical and numeric profile features and a brute-force nearest-neighbour index with cosine distance to identify similar learners. Similarity-weighted votes over their recorded next-course labels are used to rank candidate courses. The current evaluation reports a Top-5 hit rate of 53.15%, compared with 45.45% for the popularity baseline, on a holdout set of 143 learners. These results indicate a modest improvement on this dataset and should not be interpreted as proof of general real-world effectiveness.

The dashboard also presents dataset information, exploratory data-analysis charts, recommendation results, and an explanation of the available data limitations. The project demonstrates an end-to-end workflow involving data validation, feature engineering, nearest-neighbour recommendation, evaluation, and interactive deployment.

INDEX

- ACKNOWLEDGEME
- ABSTRACT
- LIST OF SCREENSHOTS

1. INTRODUCTION…… 5-6
2. LITERATURE SURVEY……6-7
3. PROBLEM STATEMENT AND OBJECTIVES……7-8
4. DATASET DESCRIPTION AND PREPARATION……9-12
5. SYSTEM ANALYSIS AND DESIGN……13-15
6. METHODOLOGY AND RECOMMENDATION ALGORITHM……16-17
7. IMPLEMENTATION AND APPLICATION WORKFLOW……18-19
8. EXPLORATORY DATA ANALYSIS AND VISUAL RESULTS…….19-20
9. TRAINING, TESTING AND EVALUATION……...21
10. LIMITATIONS AND RESPONSIBLE INTERPRETATION…….22
11. CONCLUSION AND FUTURE SCOPE……..22-23
12. REFERENCES……..23

INTRODUCTION

1.1 Background

Digital learning environments offer learners a wide range of courses. Choosing a suitable next course can be difficult when learners have different levels, preferences, completed-course histories, study hours, and performance scores.

A recommender system can help rank possible next courses by identifying patterns among learners with similar profile.

1.2 Project Overview

The Learning Recommender System is a data-driven application that compares a learner profile with profiles in a historical dataset. It identifies a neighbourhood of similar learners and uses their recorded next-course recommendations to generate a ranked list of course codes.

1.3 Scope

Accepts an existing learner ID or a manually entered learner profile.
Uses course history, learning level, learning preference, study hours, and performance score as profile features.
Finds similar learners using cosine distance.
Ranks next-course codes using similarity-weighted voting.
Displays match scores and short explanations in a Streamlit dashboard.
Provides dataset audit information, EDA charts, and model evaluation metrics.

1.4 Technologies Used

Area	Technology / Component
Programming language	Python
Dashboard	Streamlit
Machine learning	scikit-learn
Feature transformation	DictVectorizer
Similarity search	NearestNeighbors with cosine distance
Model persistence	joblib
Data handling	CSV / pandas
Visualisation	Streamlit charts and plotting libraries

1.5 Project Structure

The project separates dashboard presentation, data preparation, feature engineering, validation, exploratory analysis, recommendation logic, and model training into dedicated modules. This makes the application easier to maintain and retrain.

2. LITERATURE SURVEY

Recommender systems are commonly used to help users discover relevant items from a larger catalogue. Collaborative-filtering approaches use patterns in user-item interactions or similarities between users to estimate which items may be relevant to a target user. Neighbour-based methods are a practical form of collaborative filtering: they identify users with similar profiles or histories and aggregate their observed preferences.

This project applies a neighbour-based approach to learner profiles. Unlike a content catalogue recommender, it does not compare course descriptions or learning-resource metadata. Instead, it uses the learner attributes available in the supplied CSV and the recorded Next_Course_Recommendation target.

Feature representation is important because categorical and numeric variables must be represented consistently for similarity calculations. DictVectorizer converts dictionary-style profile features into a numerical feature matrix. Cosine distance is then used to locate nearby learner profiles. Evaluation on a held-out subset provides an estimate of how well the ranked recommendations recover the recorded next-course label.

The literature-informed design is intentionally modest: the system demonstrates a transparent nearest-neighbour ranking method reports measured performance rather than claiming that recommendations guarantee improved learning outcomes.

3. PROBLEM STATEMENT AND OBJECTIVES

3.1 Problem Statement

Learners may find it difficult to identify an appropriate next course from their current learning profile. The project addresses this by building a machine-learning system that uses historical learner data to rank likely next-course codes for a new or existing learner profile.

3.2 Objectives

Load and validate learner-profile data from a CSV file.

Convert learner attributes and course history into a consistent feature representation.

Identify similar learners using cosine-distance nearest-neighbour search.

Generate ranked next-course recommendations using similarity-weighted votes.

Exclude the learner's current course and already completed courses from candidates.

Evaluate ranking performance against a simple popularity baseline.

Present learner inputs, dataset information, EDA, and recommendations in a Streamlit dashboard.

3.3 Existing and Proposed System

Column	Type / Format	Purpose
Learner_ID	Unique identifier	Existing-learner lookup and optional self-match exclusion
Current_Course	Course code	Profile feature and recommendation exclusion
Completed_Courses	Comma-separated course codes	Course-history features and candidate exclusion
Learning_Level	Category	Profile feature
Learning_Preference	Category	Profile feature
Time_Spent_Hours	Numeric, 0–100	Scaled profile feature and profile comparison
Performance_Score	Numeric, 0–100	Scaled profile feature and profile comparison
Next_Course_Recommendation	Course code	Recorded supervision target

4. DATASET DESCRIPTION AND PREPARATION

4.1 Dataset Source

The current project uses data/learning_path_dataset.csv. This is the actual source specified by the project documentation. The project is not built on the Open University Learning Analytics Dataset (OULAD).

4.2 Required Dataset Columns

Column	Type / Format	Purpose
Learner_ID	Unique identifier	Existing-learner lookup and optional self-match exclusion
Current_Course	Course code	Profile feature and recommendation exclusion
Completed_Courses	Comma-separated course codes	Course-history features and candidate exclusion
Learning_Level	Category	Profile feature
Learning_Preference	Category	Profile feature
Time_Spent_Hours	Numeric, 0–100	Scaled profile feature and profile comparison
Performance_Score	Numeric, 0–100	Scaled profile feature and profile comparison
Next_Course_Recommendation	Course code	Recorded supervision target

4.3 Data Validation and Cleaning

Reads the CSV and verifies that all required columns are present.

Removes surrounding whitespace from string values.

Converts Time_Spent_Hours and Performance_Score to numeric values.

Rejects missing values, blank required strings, and duplicate learner IDs.

Checks that study hours and performance scores fall within the 0–100 range.

Parses comma-separated completed course codes into a normalized list.

4.4 Filtering Trainable Rows

Before training, filter_trainable_rows excludes a record when its recorded next-course target is the learner's current course or is already present in the learner's completed-course list. The current saved metrics report 286 excluded records and 714 retained records from the original 1,000-row dataset.

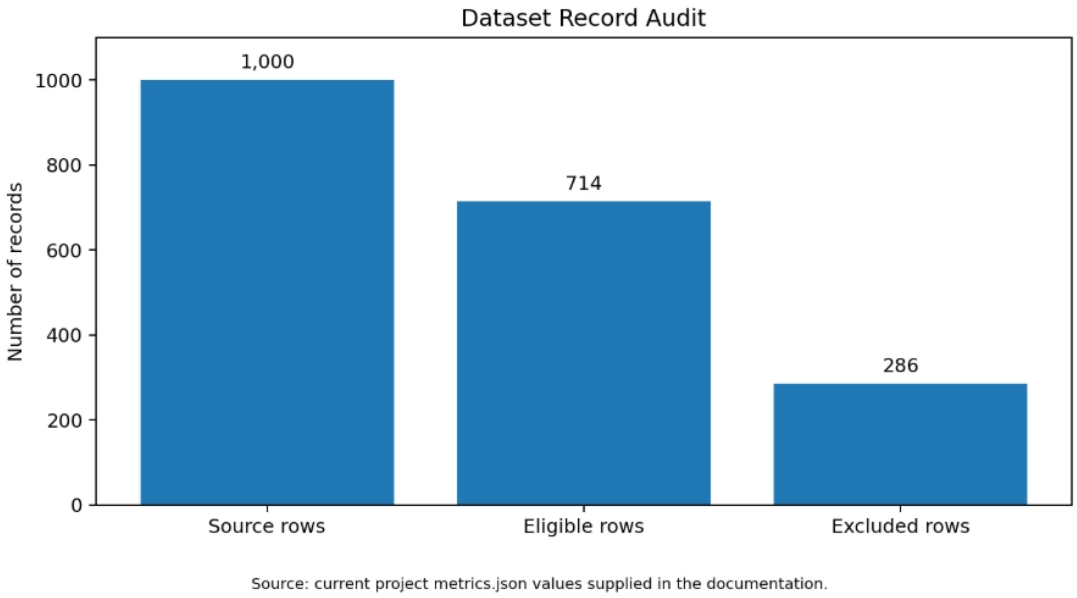


Figure 4.1 – Dataset record audit based on the current saved project metrics.

The chart shows record counts only. Detailed distributions for learning level, learning preference, score ranges, study-hour ranges, and course frequencies require the actual CSV values and are not fabricated in this report.

4.5 Feature Preparation

Feature group	Representation
Current course	Categorical indicator
Learning level	Categorical indicator
Learning preference	Categorical indicator
Completed courses	One binary history indicator per course
Study hours	Numeric value divided by 100
Performance score	Numeric value divided by 100

5. SYSTEM ANALYSIS AND DESIGN

5.1 Functional Requirements

The dashboard must load the configured CSV dataset.

The user must be able to select an existing learner or enter a manual profile.

The system must validate the profile before generating recommendations.

The model must identify similar learners while excluding the selected learner from their own neighbour list.

The system must remove the current and completed courses from candidate recommendations.

The dashboard must show ranked course codes, match scores, and supporting reasons.

5.2 Non-Functional Requirements

Maintainable modular Python source code.

Consistent feature generation between training and inference.

Transparent reporting of evaluation metrics and limitations.

Clear validation messages and readable dashboard outputs.

Reproducible model training using the documented random state.

5.3 System Architecture

The architecture connects the Streamlit interface with profile validation, feature generation, the saved recommendation bundle, the source CSV, and the recommendation-ranking process. The training script creates the reusable model bundle and metrics file.

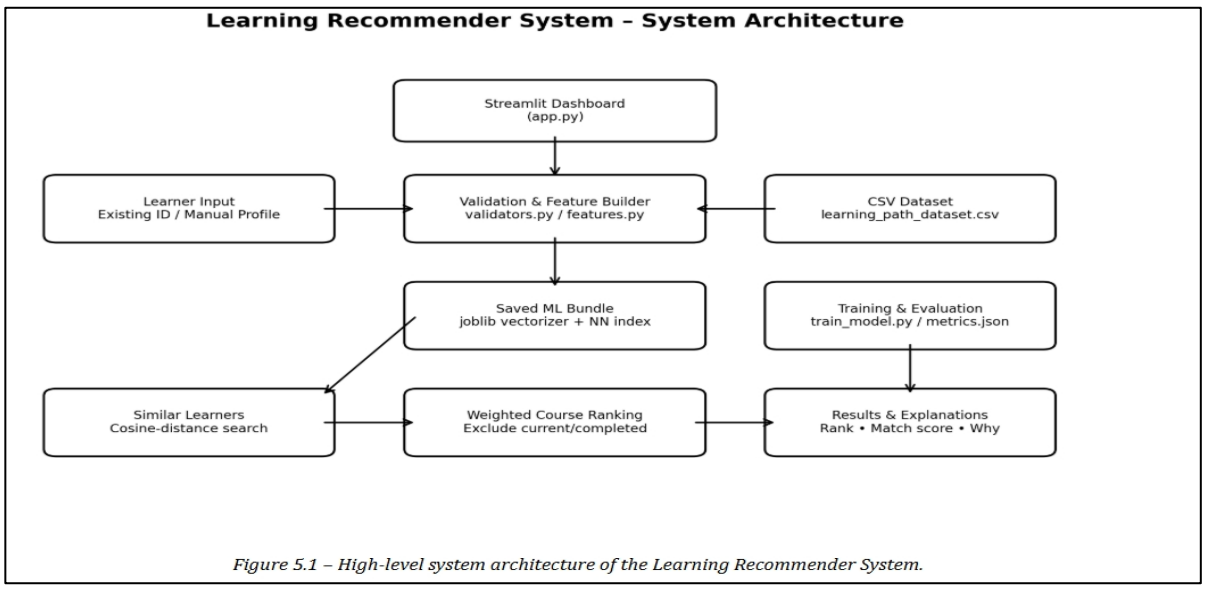


Figure 5.1 – High-level system architecture of the Learning Recommender System.

5.4 Main Modules

Module	Responsibility
Student Input	Existing learner selection or manual profile entry
Data Loading	CSV reading, schema checks, cleaning, and profile conversion
Feature Builder	Creates consistent categorical, history, and scaled numeric features
Nearest Neighbour Engine	Finds similar learner profiles using cosine distance
Ranking Engine	Aggregates similarity-weighted votes and filters invalid candidates
Evaluation	Computes ranking metrics and compares with popularity baseline
Streamlit Dashboard	Displays inputs, dataset status, EDA, ranked recommendations, and explanations

6. METHODOLOGY AND RECOMMENDATION ALGORITHM

6.1 Feature Vectorisation

The feature builder converts each learner into a dictionary containing categorical indicators, completed-course history indicators, and scaled numeric values. DictVectorizer transforms these dictionaries into a numerical matrix. The same feature-building logic is used during training and when a live learner profile is submitted.

6.2 Nearest-Neighbour Search

The recommender fits a brute-force nearest-neighbour index using cosine distance. The default neighbourhood size is 20 learners. For an existing learner lookup, the selected learner's own ID is excluded from the returned neighbours so that the learner does not match with themselves.

6.3 Similarity-Weighted Voting

Each neighbour contributes a vote to the course code recorded in Next_Course_Recommendation. A neighbour's vote is weighted by its similarity to the target learner. Candidate courses that match the learner's current course or completed-course history are removed. Remaining candidates are ordered by their accumulated vote weight.

6.4 Match Score and Explanation

Match Score: accumulated vote weight for a course divided by the total similarity weight of the selected neighbours.
Why Recommended: number of similar learners who selected the course and their average performance score.

6.5 RECOMMENDATION PROCESS

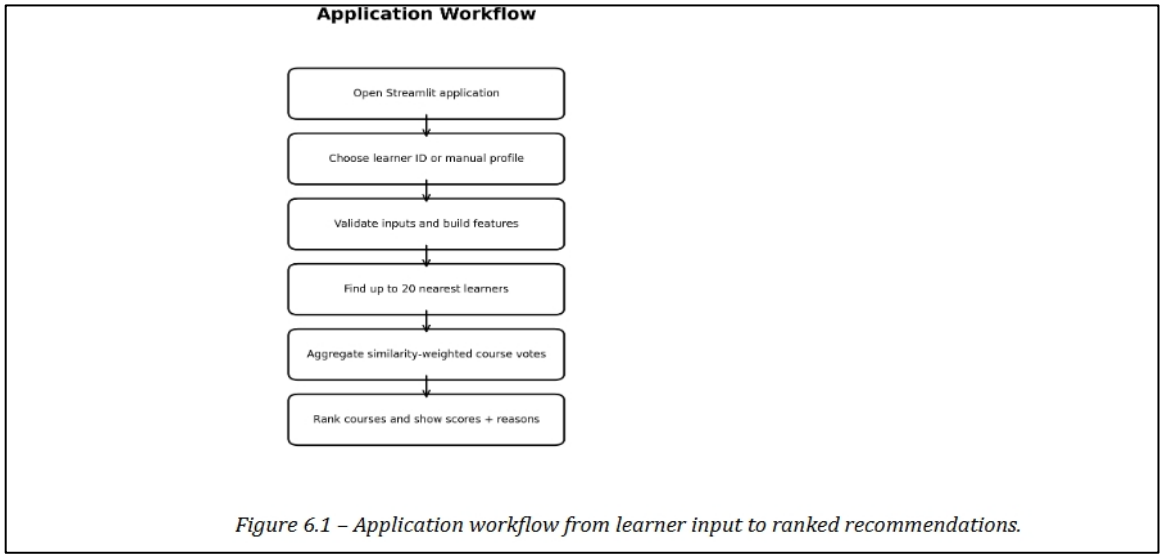


Figure 6.1 – Application workflow from learner input to ranked recommendations.

7. IMPLEMENTATION AND APPLICATION WORKFLOW

7.1 Dashboard Sections

Section	What the user sees
1. Student Input	Existing learner ID or manual profile; current course, learning level, preference, completed courses, study hours, performance score, and recommendation count (3–10).
2. Loading and Processing	Dataset status, record and course counts, excluded target rows, feature-processing information, and a data sample.
3. EDA Graphs	Score distribution, learning-preference counts, study hours versus performance, and recorded next-course label counts.
4. Recommendation Output	Ranked course codes, match scores, match-score chart, and comparison of the target learner with nearest learners.
5. Recommended Resources	Explains that the CSV contains course codes only and lists recommended course codes when results are available.

7.2 Navigation and Validation

A sticky navigation bar links to the numbered dashboard sections. Validation messages are shown in the Student Input section. The recommendation count can be selected from 3 to 10.

7.3 Training the Model

From the project root, run the following command:
python train_model.py
Training uses a stratified 80/20 split of eligible rows with random_state=42. The script evaluates recommendations on the holdout set, calculates ranking metrics and a popularity baseline, and then fits the saved recommendation bundle on all eligible rows.

7.4 Generated Model Files

File	Description
models/recommender_model.joblib	Vectorizer, nearest-neighbour index, training profiles, targets, and learner IDs.
models/metrics.json	Dataset counts, split information, model metrics, and baseline metrics.

7.5 Running the Dashboard

Install dependencies, train the model, and launch Streamlit:

python -m pip install -r requirements.txt

python train_model.py

streamlit run app.py

For Windows PowerShell, create and activate a virtual environment:

py -m venv .venv

.\.venv\Scripts\Activate.ps1

For macOS or Linux, use:

python3 -m venv .venv

source .venv/bin/activate

8. EXPLORATORY DATA ANALYSIS AND VISUAL RESULTS

8.1 EDA Views Included in the Dashboard

The dashboard is designed to display four EDA views: performance-score distribution, learning-preference counts, study hours versus performance, and counts of recorded next-course recommendation labels. These plots are generated from the loaded dataset at runtime.

The exact category counts, score distribution, and study-hour relationship are not included in the project facts supplied for this report. Therefore, no synthetic category or score values have been added. The dashboard's runtime EDA charts should be captured from the running application and inserted here as final evidence.

8.2 Dataset Record Distribution

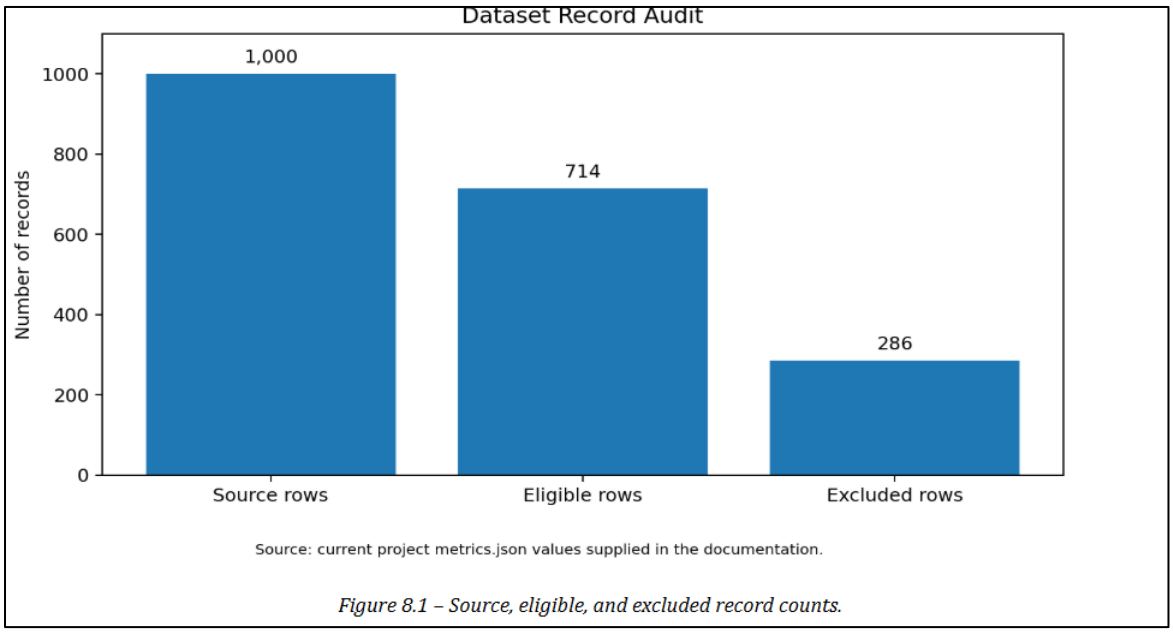


Figure 8.1 – Source, eligible, and excluded record counts.

8.3 Recommendation Performance Visualisation

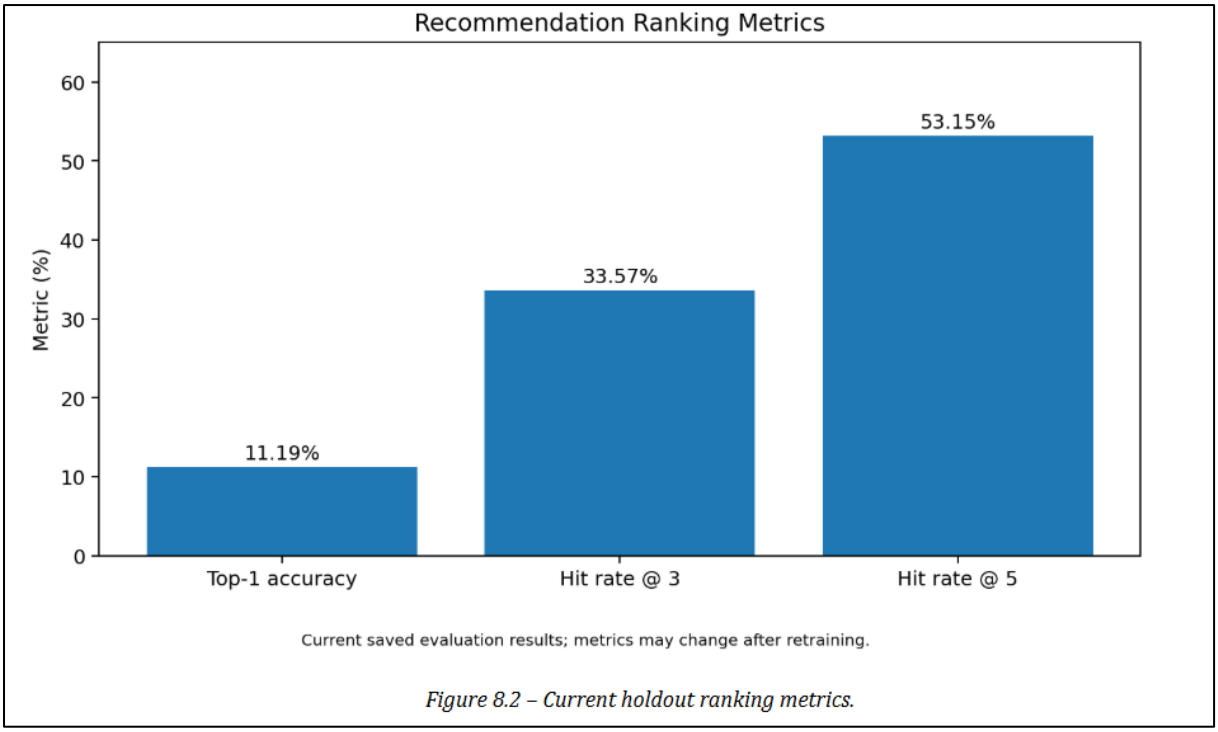


Figure 8.2 – Current holdout ranking metrics.

8.4 Model and Baseline Comparison

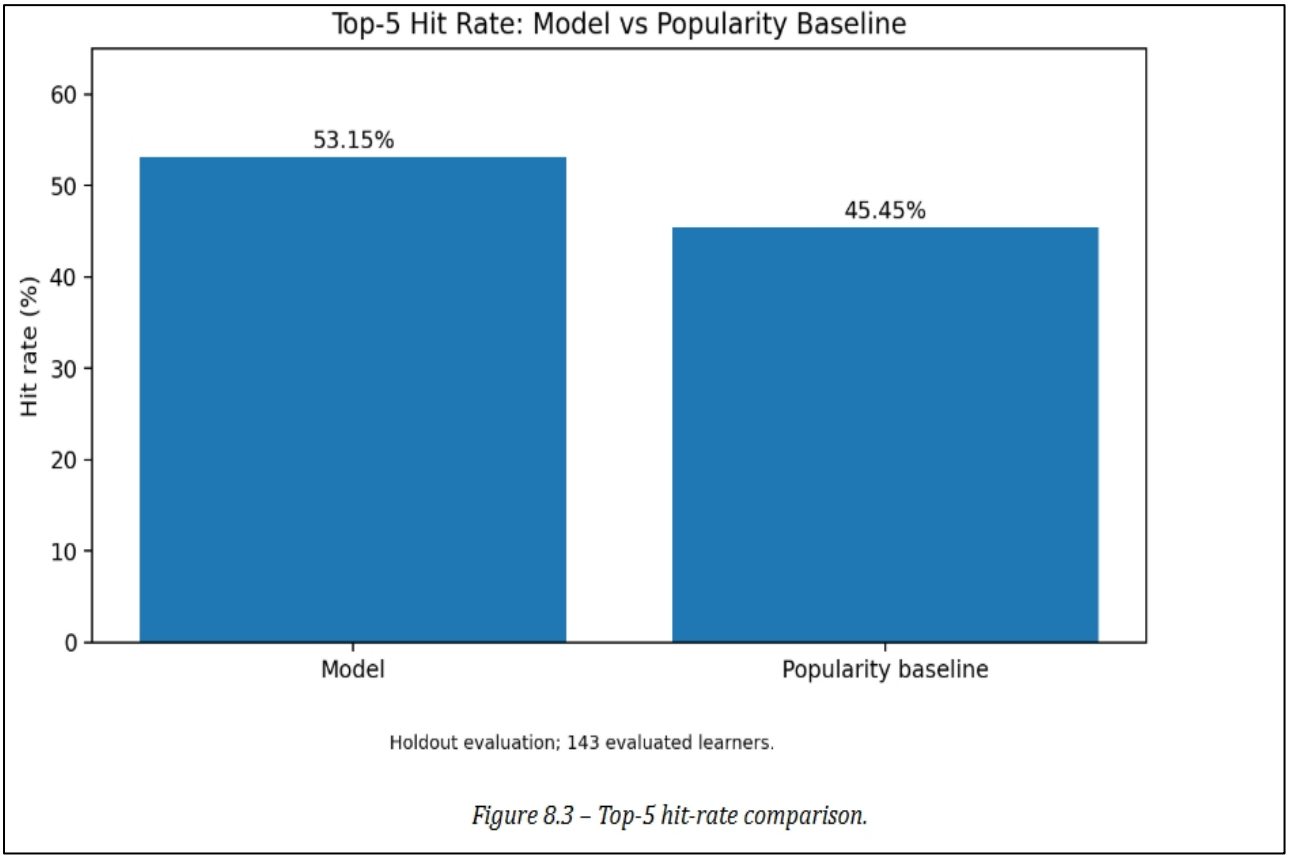


Figure 8.3 – Top-5 hit-rate comparison.

8.5 Application Screenshots to Insert

The following figures reproduce the supplied screenshots from the running Streamlit dashboard. They document the actual student input, data processing, exploratory analysis, and recommendation output.

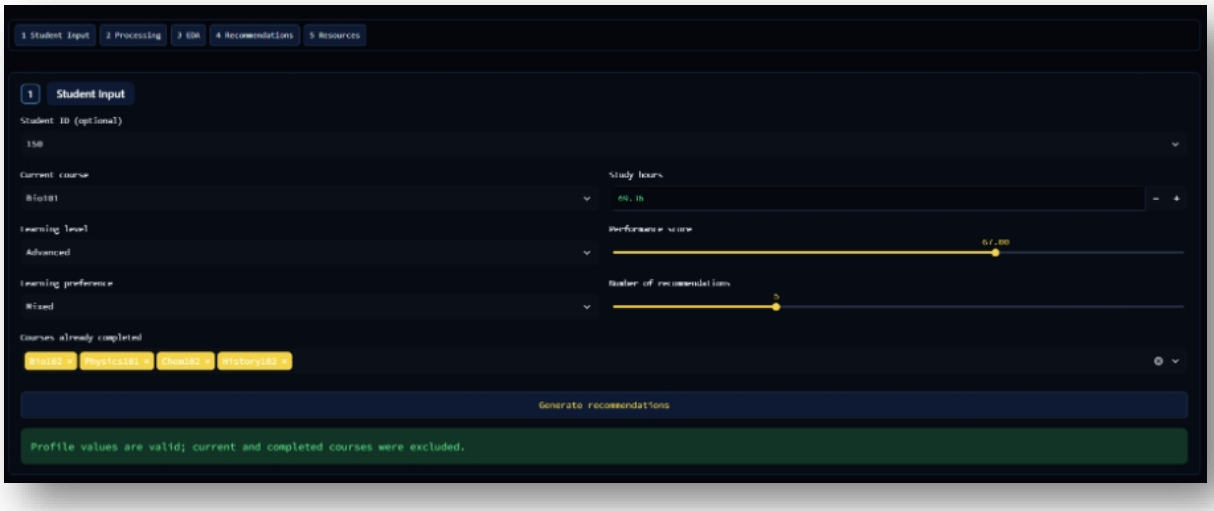


Figure 8.4 – Student Input section

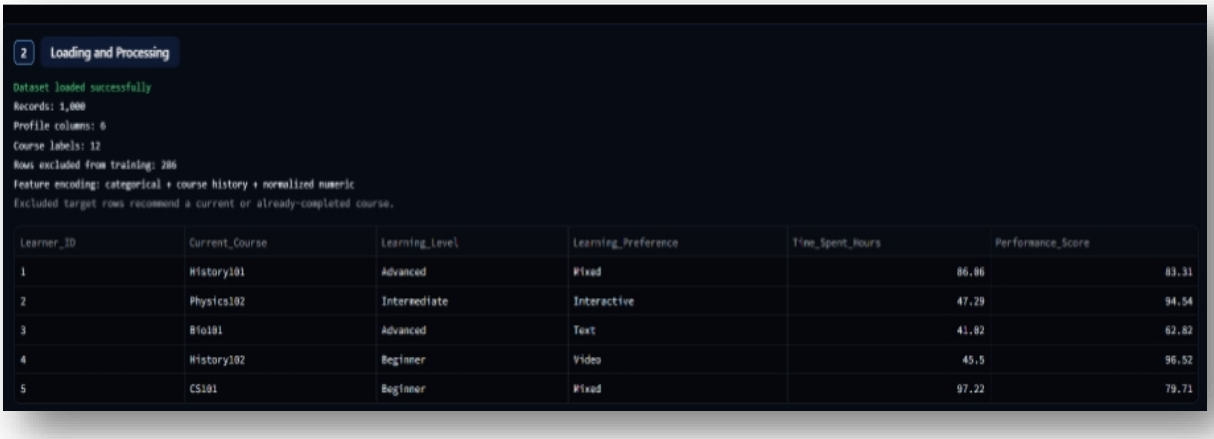


Figure 8.5 – Loading and Processing section

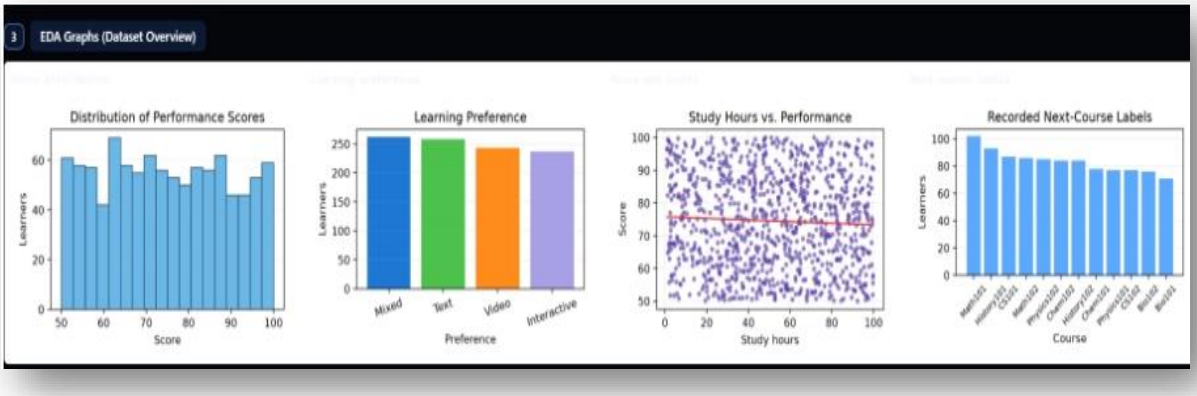


Figure 8.6 – EDA Graphs section

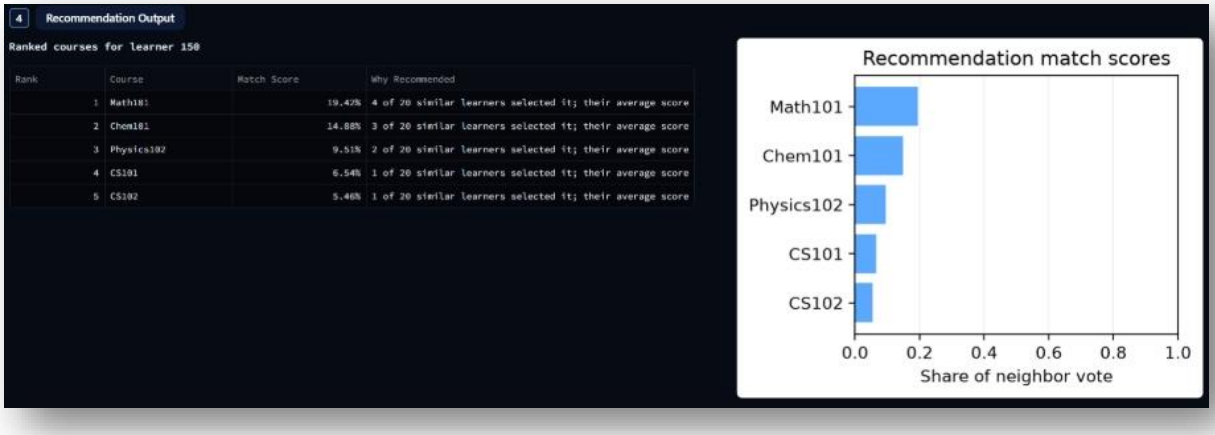


Figure 8.7 – Recommendation Output section

9. TRAINING, TESTING AND EVALUATION

9.1 Training Configuration

Parameter	Current configuration
Train/test split	Stratified 80/20
Random state	42
Eligible records	714
Excluded records	286
Holdout evaluation learners	143
Default neighbour count	20
Evaluation recommendation count	5

9.2 Current Saved Evaluation Results

The current Top-5 hit rate is approximately 7.7 percentage points higher than the popularity baseline. This is a modest improvement on the current dataset. It should not be presented as evidence of broad real-world effectiveness.

Metric	Model	Popularity baseline
Top-1 accuracy	11.19%	10.49%
Hit rate at 3	33.57%	Not calculated
Hit rate at 5	53.15%	45.45%
Mean reciprocal rank at 5	0.2472	Not calculated

9.3 Interpretation

Top-1 accuracy measures how often the first-ranked recommendation matches the recorded next-course label.

Hit rate at K measures how often the recorded target appears among the first K recommendations.

Mean reciprocal rank at 5 rewards a correct target more when it appears closer to the top of the ranked list.

The popularity baseline provides a simple reference point for judging whether profile-based ranking adds value.

9.4 Testing Checklist

Test ID	Test scenario	Expected behaviour
TC-01	Load dataset	Required columns and records are loaded; clear error if invalid.
TC-02	Select existing learner	Profile values are loaded for the selected learner.
TC-03	Enter manual profile	Valid profile is accepted and transformed using shared features.
TC-04	Invalid numeric value	Validation rejects values outside the allowed range.
TC-05	Existing learner self-match	Selected learner ID is excluded from its neighbour list.
TC-06	Candidate filtering	Current and completed courses are excluded from recommendations.
TC-07	Recommendation count	Dashboard returns the selected number, within the supported 3–10 range, where candidates permit.
TC-08	Missing model bundle	Application reports that training is required before recommendations can run.

The table describes recommended functional test cases based on the documented design. Mark each case Pass/Fail only after executing it in the actual application.

10. LIMITATIONS AND RESPONSIBLE INTERPRETATION

The system recommends course codes, not links to courses, quizzes, videos, or other learning materials.

The target is the dataset's recorded Next_Course_Recommendation; the project does not establish that a recommended course causes better learning outcomes.

Recommendations depend on patterns in this one dataset and may not generalize to other learners or institutions.

More than one quarter of source rows are excluded because their target conflicts with the learner's current or completed course history. This rule should be reviewed if target semantics change.

Holdout metrics are modest. Retrain and re-evaluate after any dataset or feature change, and report the current metrics rather than assuming improvement.

Detailed EDA distributions must be generated from the actual CSV; they should not be inferred from aggregate row counts.

11. CONCLUSION AND FUTURE SCOPE

11.1 Conclusion

The Learning Recommender System demonstrates an end-to-end machine-learning workflow for ranking next-course recommendations from learner profile data. It validates a CSV dataset, prepares categorical and numeric features, identifies similar learners using cosine distance, and ranks course codes through similarity-weighted voting. The Streamlit dashboard brings the workflow together with learner input, dataset status, EDA views, recommendation scores, and explanations.

The current saved evaluation reports a Top-5 hit rate of 53.15%, compared with 45.45% for the popularity baseline, on 143 holdout learners. This represents a modest improvement on the supplied dataset. The result should be interpreted cautiously and rechecked after retraining or data changes.

11.2 Future Scope

Add verified course metadata such as titles, descriptions, prerequisites, and course URLs so course codes can be presented with richer information.

Use a larger and more diverse learner dataset to test generalisation across courses and learner groups.

Explore alternative recommendation methods and compare them using the same holdout protocol.

Add more robust cross-validation and confidence or coverage analysis.

Monitor recommendation quality after deployment and retrain when the underlying data changes.

12. REFERENCES

Project source dataset: data/learning_path_dataset.csv (local project data file).

Scikit-learn documentation – DictVectorizer: https://scikit-learn.org/stable/modules/generated/sklearn.feature_extraction.DictVectorizer.html

Scikit-learn documentation – Nearest Neighbors: https://scikit-learn.org/stable/modules/neighbors.html

Scikit-learn documentation – Model evaluation: https://scikit-learn.org/stable/modules/model_evaluation.html

Streamlit documentation: https://docs.streamlit.io/

Joblib documentation: https://joblib.readthedocs.io/

Python documentation: https://docs.python.org/3/

Project-specific implementation details and saved evaluation figures are based on the project documentation supplied for this report. The actual CSV and source-code files should be used to verify or update all data-derived values before final academic submission.

13. PROJECT OUTPUT SUMMARY

The supplied dashboard screenshots show a working end-to-end flow. In the illustrated example, learner 150 has current course Bio101, Advanced learning level, Mixed learning preference, 69.36 study hours, and a performance score of 67. The learner's completed courses are excluded from candidate recommendations.

The displayed top-five recommendations are Math101 (19.42%), Chem101 (14.88%), Physics102 (9.51%), CS101 (6.54%), and CS102 (5.46%). These percentages are the displayed shares of similarity-weighted neighbour votes for that example learner; they are not model accuracy.

The processing panel reports 1,000 source records, 6 profile columns, 12 course labels, and 286 rows excluded because their recorded target is the current or an already-completed course. This leaves 714 eligible rows.

The saved holdout evaluation reports 143 evaluated learners, Top-1 accuracy of 11.19%, Hit Rate@3 of 33.57%, Hit Rate@5 of 53.15%, and Mean Reciprocal Rank@5 of 0.2472. The popularity baseline has Top-1 accuracy of 10.49% and Hit Rate@5 of 45.45%. Results are dataset-specific and should be rechecked after retraining.